

Celery Tasks — Async Jobs, Queues & Beat Schedule

Tech: Celery 5.x + Redis broker + django-celery-beat for periodic tasks

Table of Contents

1. [Celery Application Setup](#)
 2. [Queue Architecture](#)
 3. [All Tasks Reference](#)
 4. [Celery Beat Schedule](#)
 5. [Task Reliability Patterns](#)
 6. [Monitoring & Alerting](#)
-

1. Celery Application Setup

```
# celery_app.py (project root)

import os
from celery import Celery
from django.conf import settings

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
                      'config.settings.production')

app = Celery('bsecure')

# Load config from Django settings (CELERY_* keys)
app.config_from_object('django.conf:settings', namespace='CELERY')

# Auto-discover tasks in all INSTALLED_APPS
app.autodiscover_tasks()

@app.task(bind=True)
def debug_task(self):
    print(f'Request: {self.request!r}')
```

```

# config/settings/base.py (Celery configuration)

import os
from kombu import Queue, Exchange

# Broker and result backend (both Redis)
CELERY_BROKER_URL = env('REDIS_URL')
CELERY_RESULT_BACKEND = env('REDIS_URL')

# Serialization
CELERY_TASK_SERIALIZER = 'json'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_RESULT_SERIALIZER = 'json'

# Timezone
CELERY_TIMEZONE = 'Asia/Kolkata'
CELERY_ENABLE_UTC = True

# Task acknowledgement: ack after execution (not before) for
# reliability
CELERY_TASK_ACKS_LATE = True
CELERY_WORKER_PREFETCH_MULTIPLIER = 1 # Process one task at a
# time per worker slot

# Task time limits
CELERY_TASK_SOFT_TIME_LIMIT = 300
# 5 min soft limit – task gets SoftTimeLimitExceeded
CELERY_TASK_TIME_LIMIT = 360 # 6 min hard limit – task is
# killed

# Queue definitions
CELERY_TASK_QUEUES = (
    Queue('high_priority', Exchange('high_priority'),
        routing_key='high_priority'),
    Queue('default', Exchange('default'),
        routing_key='default'),
    Queue('low_priority', Exchange('low_priority'),
        routing_key='low_priority'),
    Queue('scheduled', Exchange('scheduled'),
        routing_key='scheduled'),
)
CELERY_TASK_DEFAULT_QUEUE = 'default'

# Retry policy
CELERY_TASK_MAX_RETRIES = 3
CELERY_TASK_RETRY_BACKOFF = True
CELERY_TASK_RETRY_BACKOFF_MAX = 600 # Max 10 min between retries

# Store task results (for monitoring)
CELERY_TASK_TRACK_STARTED = True
CELERY_RESULT_EXPIRES = 86400 # Keep results for 24 hours

# Beat scheduler

```

```

CELERY_BEAT_SCHEDULER =
    'django_celery_beat.schedulers:DatabaseScheduler'

# Routing: which tasks go to which queues
CELERY_TASK_ROUTES = {
    # High priority – SOS, OTP, booking requests
    'notifications.tasks.send_otp_sms': {'queue':
        'high_priority'},
    'notifications.tasks.notify_guard_assigned': {'queue':
        'high_priority'},
    'notifications.tasks.notify_emergency_contacts': {'queue':
        'high_priority'},
    'notifications.tasks.notify_guard_of_user_sos': {'queue':
        'high_priority'},
    'bookings.tasks.broadcast_booking_request': {'queue':
        'high_priority'},
    'sos.tasks.schedule_sos_escalation': {'queue':
        'high_priority'},
    'payments.tasks.process_razorpay_event': {'queue':
        'high_priority'},

    # Default – standard notifications, payments
    'notifications.tasks.notify_session_completed': {'queue':
        'default'},
    'notifications.tasks.send_wallet_topup_notification':
        {'queue': 'default'},
    'payments.tasks.process_session_payment': {'queue':
        'default'},
    'payments.tasks.execute_payout': {'queue':
        'default'},
    'sos.tasks.monitor_session_checkins': {'queue':
        'default'},
    'sos.tasks.check_guard_offline_sessions': {'queue':
        'default'},

    # Low priority – PDF generation, analytics
    'payments.tasks.generate_invoice_pdf': {'queue':
        'low_priority'},
    'notifications.tasks.send_invoice_email': {'queue':
        'low_priority'},
    'analytics.tasks.aggregate_daily_stats': {'queue':
        'low_priority'},
    'analytics.tasks.cleanup_old_location_snapshots': {'queue':
        'low_priority'},

    # Scheduled – periodic maintenance
    'payments.tasks.process_weekly_payouts': {'queue':
        'scheduled'},
    'authentication.tasks.cleanup_expired_tokens': {'queue':
        'scheduled'},
    'guards.tasks.check_document_expiry': {'queue':
        'scheduled'},
}

```

2. Queue Architecture

Queue: high_priority

Workers: 4 (dedicated, never starved)

Tasks: OTP SMS, SOS alerts, booking broadcast, emergency notifications

Max retries: 5

Retry delay: 5s, 10s, 30s

Queue: default

Workers: 4

Tasks: Standard notifications, payment processing, check-in monitoring

Max retries: 3

Retry delay: 30s, 60s, 120s

Queue: low_priority

Workers: 2

Tasks: PDF generation, email with attachments, analytics aggregation

Max retries: 2

Retry delay: 60s

Queue: scheduled

Workers: 1 (beat tasks only – one instance)

Tasks: Nightly payouts, token cleanup, document expiry checks

Worker startup commands:

High priority workers (production)

```
celery -A celery_app worker \
  -Q high_priority \
  -c 4 \
  -n worker-high@%h \
  --loglevel=info \
  --logfile=/var/log/celery/high.log
```

Default workers

```
celery -A celery_app worker \
  -Q default \
  -c 4 \
  -n worker-default@%h \
  --loglevel=info
```

Low priority workers

```
celery -A celery_app worker \
  -Q low_priority \
  -c 2 \
  -n worker-low@%h \
  --loglevel=info
```

Beat scheduler (only ONE instance in the entire cluster)

```
celery -A celery_app beat \
    --scheduler django_celery_beat.schedulers:DatabaseScheduler \
    --loglevel=info \
    --logfile=/var/log/celery/beat.log
```

3. All Tasks Reference

Authentication App

```
# apps/authentication/tasks.py

from celery import shared_task

@shared_task(queue='scheduled',
             name='authentication.cleanup_expired_tokens')
def cleanup_expired_tokens():
    """
    Nightly: clean up expired OTP tokens and JWT blacklist
    entries.
    Prevents these tables from growing unboundedly.
    """
    from django.utils import timezone
    from apps.authentication.models import OTPToken
    from rest_framework_simplejwt.token_blacklist.models import OutstandingToken, BlacklistedToken

    # Delete OTP tokens older than 24 hours
    deleted_otps, _ = OTPToken.objects.filter(
        created_at__lt=timezone.now() - timedelta(hours=24)
    ).delete()

    # Delete blacklisted JWT tokens that have expired
    deleted_jwts = BlacklistedToken.objects.filter(
        token__expires_at__lt=timezone.now()
    ).delete()[0]

    return f'Cleaned up {deleted_otps} OTPs, {deleted_jwts} blacklisted JWTs'
```

Bookings App

```
# apps/bookings/tasks.py

from celery import shared_task
from datetime import timedelta
import logging

logger = logging.getLogger(__name__)
```

```

@shared_task(
    bind=True, max_retries=1,
    queue='high_priority',
    name='bookings.broadcast_booking_request'
)
def broadcast_booking_request(self, booking_id: str, radius_km:
    int = 5):
    """
    Broadcast a booking request to all nearby available guards.

    Algorithm:
    1. Find available guards within radius_km
    2. Send push notification to each (booking request appears in
        guard app)
    3. If no guards found within timeout, expand radius and re-
        broadcast
    4. If no guards found in 10km within 5 minutes, expire booking
    """

    from apps.bookings.models import Booking, BookingBroadcast
    from apps.guards.models import GuardProfile
    from apps.notifications.services.push import FCMService
    from django.contrib.gis.geos import Point
    from django.contrib.gis.measure import D
    from django.utils import timezone

    try:
        booking =
            Booking.objects.select_related('user').get(id=booking_id)
    except Booking.DoesNotExist:
        return

    if booking.status not in ('REQUESTED', 'BROADCAST'):
        return # Already handled

    # Transition to BROADCAST state
    if booking.status == 'REQUESTED':
        booking.start_broadcast()
        booking.save()

    user_location = Point(
        float(booking.service_longitude),
        float(booking.service_latitude),
        srid=4326
    )

    # Find nearby available guards who haven't already received
    # this request
    already_sent = BookingBroadcast.objects.filter(
        booking=booking
    ).values_list('guard_id', flat=True)

```

```

nearby_guards = GuardProfile.objects.filter(
    is_online=True,
    verification_status='ACTIVE',
    guard_type=booking.guard_type_requested,
    current_location__distance_lte=(user_location,
    D(km=radius_km)),
).exclude(id__in=already_sent).order_by('?')[:15]  # Max 15
guards per broadcast

if not nearby_guards.exists():
    if radius_km < 10:
        # Expand radius and retry after 90 seconds
        logger.info(f'Booking {booking_id}: no guards in
        {radius_km}km, expanding to {radius_km + 5}km')
        broadcast_booking_request.apply_async(
            args=[booking_id, radius_km + 5],
            countdown=90,
            queue='high_priority',
        )
    else:
        # Give up – expire the booking
        expire_booking.apply_async(
            args=[booking_id],
            countdown=30,
            queue='high_priority',
        )
    return

# Send requests to found guards
for guard in nearby_guards:
    BookingBroadcast.objects.get_or_create(
        booking=booking,
        guard=guard,
        defaults={'broadcast_radius_km': radius_km}
    )
    if guard.user.fcm_token:
        FCMService.send_to_device(
            fcm_token=guard.user.fcm_token,
            title='New Booking Request!',
            body=f'{booking.service_type} –
            {booking.guard_type_requested} – accept in 30 sec',
            data={
                'booking_id': str(booking_id),
                'action': 'NEW_BOOKING_REQUEST',
                'service_type': booking.service_type,
                'duration_hours': str(
                    (booking.scheduled_end –
                    booking.scheduled_start).seconds // 3600
                ),
            },
            notification_type='NEW_BOOKING_REQUEST',
        )

```

```

# Schedule expiry if no guard accepts within 5 minutes
expire_unaccepted_booking.apply_async(
    args=[booking_id],
    countdown=300, # 5 minutes
    queue='high_priority',
)

return f'Broadcast to {len(nearby_guards)} guards within
{radius_km}km'

@shared_task(queue='high_priority',
             name='bookings.expire_unaccepted')
def expire_unaccepted_booking(booking_id: str):
    """Called 5 min after broadcast if booking still not
    accepted."""
    from apps.bookings.models import Booking
    from apps.notifications.tasks import notify_no_guard_found

    try:
        booking = Booking.objects.get(id=booking_id)
    except Booking.DoesNotExist:
        return

    if booking.status == 'BROADCAST':
        booking.expire()
        booking.save()
        notify_no_guard_found.delay(booking_id)

@shared_task(queue='high_priority',
             name='bookings.expire_booking')
def expire_booking(booking_id: str):
    """Expire a booking that could not be matched."""
    from apps.bookings.models import Booking

    try:
        booking = Booking.objects.get(id=booking_id,
                                       status='BROADCAST')
        booking.expire()
        booking.save()
    except Booking.DoesNotExist:
        pass

@shared_task(queue='default',
             name='bookings.send_session_reminder')
def send_session_reminder(booking_id: str):
    """Send reminder push 1 hour before scheduled advance
    booking."""
    from apps.bookings.models import Booking
    from apps.notifications.tasks import notify_booking_reminder

    try:

```

```

        booking = Booking.objects.get(id=booking_id,
        status='ACCEPTED')
        notify_booking_reminder.delay(booking_id)
    except Booking.DoesNotExist:
        pass

```

Guards App

apps/guards/tasks.py

```

from celery import shared_task

@shared_task(queue='scheduled',
              name='guards.check_document_expiry')
def check_document_expiry():
    """
    Daily: find guards with documents expiring within 30 days.
    Send reminder notifications and flag in admin panel.
    """

    from apps.guards.models import GuardDocument
    from apps.notifications.tasks import \
        notify_document_expiry_reminder
    from django.utils import timezone
    from datetime import timedelta

    thirty_days_out = timezone.now().date() + timedelta(days=30)

    expiring_docs = GuardDocument.objects.filter(
        status='APPROVED',
        expiry_date__lte=thirty_days_out,
        expiry_date__gte=timezone.now().date(),
        expiry_reminder_sent=False,
    ).select_related('guard__user')

    for doc in expiring_docs:
        days_left = (doc.expiry_date - timezone.now().date()).days
        notify_document_expiry_reminder.delay(str(doc.id),
        days_left)
        doc.expiry_reminder_sent = True
        doc.save(update_fields=['expiry_reminder_sent'])

    return f'Sent expiry reminders for {expiring_docs.count()}
    documents'

```

Analytics App

apps/analytics/tasks.py

```

from celery import shared_task
from datetime import timedelta
from django.utils import timezone

```

```

@shared_task(queue='low_priority',
             name='analytics.aggregate_daily_stats')
def aggregate_daily_stats():
    """
    Nightly at midnight: aggregate stats for the previous day.
    Populates DailyStats table used by admin dashboard for fast
    reads.
    """
    from apps.analytics.models import DailyStats
    from apps.bookings.models import Booking
    from apps.users.models import UserProfile
    from apps.guards.models import GuardProfile
    from apps.sos.models import SOSAlert, Incident
    from apps.payments.models import Transaction
    from django.db.models import Sum, Count

    yesterday = timezone.now().date() - timedelta(days=1)

    # Avoid duplicate creation
    if DailyStats.objects.filter(date=yesterday).exists():
        return f'Stats for {yesterday} already exist'

    bookings = Booking.objects.filter(created_at__date=yesterday)
    completed = bookings.filter(status='COMPLETED')

    revenue = Transaction.objects.filter(
        transaction_type='BOOKING_DEBIT',
        created_at__date=yesterday,
        status='SUCCESS',
    ).aggregate(total=Sum('amount'))['total'] or 0

    platform_fees = completed.aggregate(
        total=Sum('platform_fee')
    )['total'] or 0

    guard_earnings = completed.aggregate(
        total=Sum('guard_earnings')
    )['total'] or 0

    DailyStats.objects.create(
        date=yesterday,
        total_bookings=bookings.count(),
        completed_bookings=completed.count(),
        cancelled_bookings=bookings.filter(status='CANCELLED').count(),
        disputed_bookings=bookings.filter(status='DISPUTED').count(),
        hourly_bookings=bookings.filter(service_type='HOURLY').count(),
        daily_bookings=bookings.filter(service_type='DAILY').count(),
        weekly_bookings=bookings.filter(service_type='WEEKLY').count(),
        monthly_bookings=bookings.filter(service_type='MONTHLY').count(),
        gross_revenue=revenue,
        platform_fees_collected=platform_fees,
    )

```

```

guard_earnings_paid=guard_earnings,
new_users=UserProfile.objects.filter(created_at__date=yesterday,
role='USER').count(),
active_users=Booking.objects.filter(
    created_at__date=yesterday
).values('user').distinct().count(),
new_guards=GuardProfile.objects.filter(created_at__date=yesterday).count(),
active_guards=Booking.objects.filter(
    status='COMPLETED',
    session_ended_at__date=yesterday
).values('guard').distinct().count(),
sos_alerts=SOSAlert.objects.filter(created_at__date=yesterday).count(),
incidents_filed=Incident.objects.filter(created_at__date=yesterday).count()
)

return f'Daily stats aggregated for {yesterday}'

@shared_task(queue='low_priority',
name='analytics.cleanup_location_snapshots')
def cleanup_old_location_snapshots():
    """
    Weekly: delete location snapshots older than 90 days.
    High-volume table – must be managed to prevent unbounded
    growth.
    """
    from apps.tracking.models import LocationSnapshot
    from django.utils import timezone
    from datetime import timedelta

    cutoff = timezone.now() - timedelta(days=90)
    deleted, _ =
        LocationSnapshot.objects.filter(timestamp__lt=cutoff).delete()
    return f'Deleted {deleted} location snapshots older than 90
    days'

```

4. Celery Beat Schedule

Configured via django-celery-beat (stored in DB, manageable via admin):

```

# config/settings/base.py

from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    # Every 5 minutes: check for guards offline during active
    # sessions
    'check-guard-offline': {
        'task': 'sos.tasks.check_guard_offline_sessions',
        'schedule': crontab(minute='*/5'),
    },
}

```

```

# Every 15 minutes: check for missed guard check-ins
'monitor-checkins': {
    'task': 'sos.tasks.monitor_session_checkins',
    'schedule': crontab(minute='*/15'),
},

# Every 30 seconds: push live stats to admin dashboard
WebSocket
'push-live-stats': {
    'task': 'admin_panel.tasks.push_live_dashboard_stats',
    'schedule': 30.0, # Every 30 seconds
},

# Daily at 11:55 PM IST: aggregate yesterday's stats
'aggregate-daily-stats': {
    'task': 'analytics.tasks.aggregate_daily_stats',
    'schedule': crontab(hour=23, minute=55),
},

# Daily at midnight: cleanup expired OTPs and JWT tokens
'cleanup-expired-tokens': {
    'task': 'authentication.tasks.cleanup_expired_tokens',
    'schedule': crontab(hour=0, minute=0),
},

# Daily at 8 AM: check document expiry (30-day warning)
'check-document-expiry': {
    'task': 'guards.tasks.check_document_expiry',
    'schedule': crontab(hour=8, minute=0),
},

# Every Friday at 10 PM IST: process weekly guard payouts
'process-weekly-payouts': {
    'task': 'payments.tasks.process_weekly_payouts',
    'schedule': crontab(hour=22, minute=0,
        day_of_week='friday'),
},

# Weekly on Sunday midnight: cleanup old location snapshots
'cleanup-location-snapshots': {
    'task': 'analytics.tasks.cleanup_location_snapshots',
    'schedule': crontab(hour=0, minute=0,
        day_of_week='sunday'),
},

# Every 5 minutes: send session reminders for upcoming
advance bookings
'send-booking-reminders': {
    'task': 'bookings.tasks.send_upcoming_booking_reminders',
    'schedule': crontab(minute='*/5'),
}

```

```
    },  
}
```

5. Task Reliability Patterns

Idempotency

All tasks must be safe to run multiple times:

```
@shared_task(queue='default')  
def process_session_payment(booking_id: str):  
    from apps.bookings.models import Booking  
    from apps.payments.models import Transaction  
  
    # Idempotency check – was this already processed?  
    already_paid = Transaction.objects.filter(  
        booking_id=booking_id,  
        transaction_type='BOOKING_DEBIT',  
        status='SUCCESS',  
    ).exists()  
  
    if already_paid:  
        logger.info(f'Payment for booking {booking_id} already  
            processed, skipping')  
        return # Safe to return – don't double-charge  
  
    # ... rest of payment logic
```

Error Handling with Retry

```
@shared_task(  
    bind=True,  
    max_retries=3,  
    default_retry_delay=30,  
    queue='default',  
)  
def send_sms_task(self, phone: str, message: str):  
    from apps.notifications.services.sms import SMSService  
    try:  
        result = SMSService.send_sms(phone, message)  
        if not result['success']:  
            raise Exception(f"SMS failed: {result['error']}")  
    except Exception as exc:  
        logger.warning(f'SMS task failed (attempt  
            {self.request.retries + 1}): {exc}')  
        raise self.retry(exc=exc, countdown=30 * (2 **  
            self.request.retries))
```

Transaction Safety

```
@shared_task(queue='default')
def process_payout(payout_id: str):
    from django.db import transaction as db_transaction

    with db_transaction.atomic():
        # Lock the payout row to prevent double-processing
        payout =
        Payout.objects.select_for_update(nowait=True).get(id=payout_id)

        if payout.status != 'PENDING':
            return # Already in progress by another worker

        payout.status = 'PROCESSING'
        payout.save()
        # ... rest of payout logic
```

6. Monitoring & Alerting

Flower (Web UI for Celery)

```
# Start Flower dashboard (accessible at :5555)
celery -A celery_app flower \
    --port=5555 \
    --basic_auth=admin:your_secure_password \
    --url_prefix=flower
```

Flower shows: - Active workers and their status - Task success/failure rates - Queue depths (backlog) - Task execution times - Failed task details

Queue Depth Monitoring (Datadog / CloudWatch)

```
# utils/monitoring.py

import redis
from django.conf import settings

def get_queue_depths() -> dict:
    """Return current task count in each queue."""
    r = redis.from_url(settings.REDIS_URL)
    return {
        'high_priority': r.llen('high_priority'),
        'default': r.llen('default'),
        'low_priority': r.llen('low_priority'),
```

```
        'scheduled': r.llen('scheduled'),  
    }
```

Alert thresholds: - high_priority queue > 100 tasks → PagerDuty alert -
default queue > 500 tasks → Slack warning - Any queue growing
continuously for 10 min → auto-scale workers

Sentry Integration

```
# Celery tasks automatically report exceptions to Sentry via the  
    integration  
# configured in production settings:  
# sentry_sdk.init(..., integrations=[CeleryIntegration()])  
  
# Failed tasks will appear in Sentry with:  
# - Full stack trace  
# - Task arguments  
# - Queue name  
# - Retry count
```